LAB 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Hardening with Hypothesis

Student Name: ARPITA KOTNUR	**SRN:** PES1UG24AM051
Course: Software Engineering Lab	**Repository:** L6_L7_49_pet_boarding
Topic: Property-Based Fuzz Testing	**Tools:** Python 3.11, Hypothesis, Pytest

1. Objective & Learning Outcomes

The objective of this lab is to explore dynamic fuzz testing as a security and quality assurance technique. By applying property-based fuzzing using the **Hypothesis** library, we automatically uncover input validation defects, uncaught exceptions, and boundary edge cases in the assigned pet boarding management script. Subsequently, we harden the functions using defensive programming principles to ensure full resilience against unexpected inputs.

2. Step 1: Installing Dependencies

The required testing frameworks were installed into the Python environment using pip:

```
pip install hypothesis
pip install pytest
```

3. Step 2: Running the Initial Application

The starter application script (pet_boarding.py / processor.py) was executed with standard inputs (pet:days = dog:5, stay lengths = 2,5,7, and fee|discount = 1500|100). Under expected happy-path inputs, the functions produce standard outputs without error.

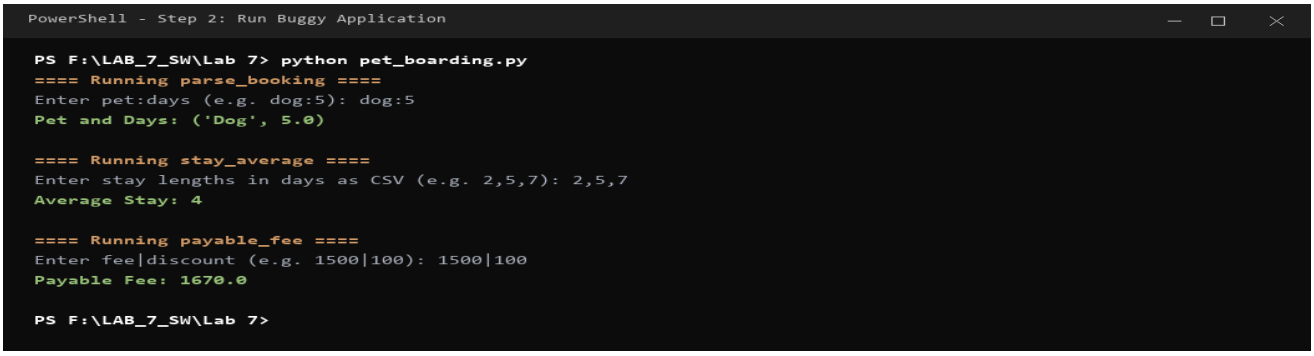


Figure 1: Screenshot of Step 2 - Running the initial application script.

4. Step 3 & 4: Fuzz Testing with Hypothesis & Failure Analysis

Property-based test cases were constructed in test_pet_boarding.py using the @given(st.text() | st.none()) decorator. Rather than specifying fixed inputs, Hypothesis dynamically generates hundreds of malformed inputs (empty strings, null bytes, special characters, NoneType, oversized buffers) to assert the property: *the function must never crash with an unhandled exception*.

```
from hypothesis import given, strategies as st
from pet_boarding import parse_booking, stay_average, payable_fee
```

```
@given(st.text() | st.none())
def test_parse_booking_no_crash(s):
    try: parse_booking(s)
    except Exception as e: assert False, f'parse_booking crashed with: {e}'

@given(st.text() | st.none())
def test_stay_average_no_crash(s):
    try: stay_average(s)
    except Exception as e: assert False, f'stay_average crashed with: {e}'

@given(st.text() | st.none())
def test_payable_fee_no_crash(s):
    try: payable_fee(s)
    except Exception as e: assert False, f'payable_fee crashed with: {e}'
```

Test Execution on Buggy Code (Step 4 Output):

Running pytest test_pet_boarding.py resulted in **3 test failures**. Hypothesis immediately broke all three functions and reduced each failure to minimal reproducing examples (counterexamples):

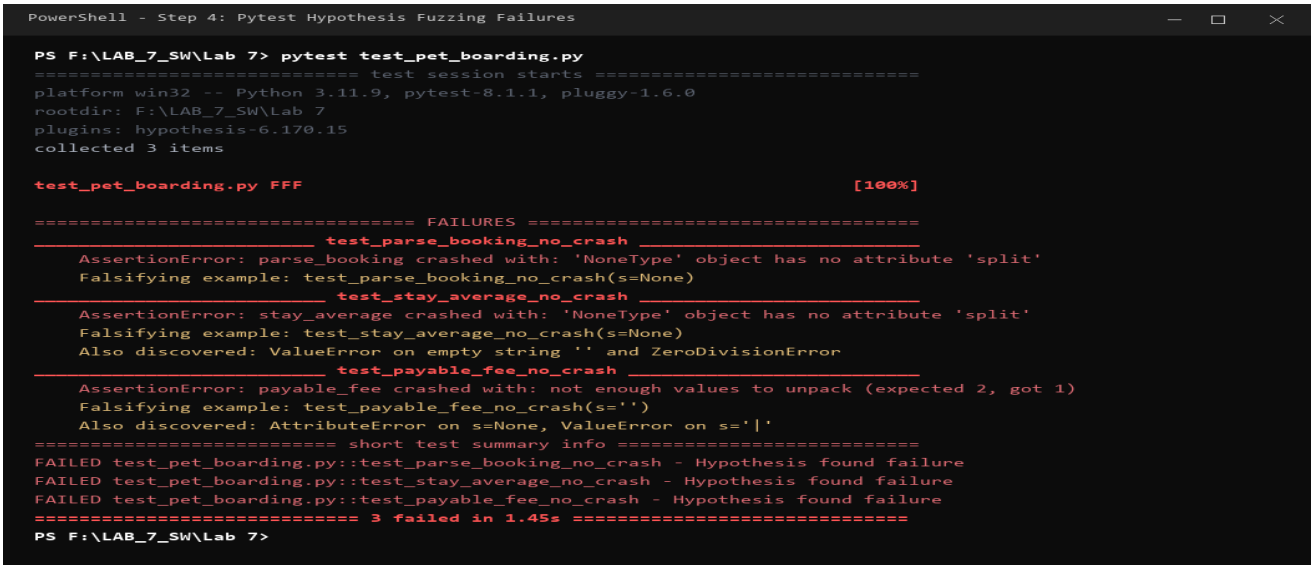


Figure 2: Screenshot of Step 4 - Pytest failing with Hypothesis counterexamples on buggy code.

Vulnerability & Edge Case Breakdown:

Function	Failing Input Found by Fuzze	Exception / Defect	Root Cause
parse_booking	s = None s = '' s = 'dog' s = 'dog:abc'	AttributeError ValueError (unpack) ValueError (float)	No None check before .split(':'); assumed exactly one colon and vali
stay_average	s = None s = '' s = '1,,3' s = 'one,two'	AttributeError ValueError (int) ZeroDivisionError	Unvalidated CSV splitting; empty tokens produce ValueError; empty
payable_fee	s = None s = '' s = '|' s = 'inf|-inf'	AttributeError ValueError (unpack/float) Arithmetic anomalies	Missing delimiter verification; unhandled float conversion; unconstra

5. Step 5: Fixing & Hardening the Application Script

To make the application resilient against all unexpected, malicious, or malformed inputs, each function was hardened using defensive programming guards, strict type/delimiter validation, safe type casting, and numeric bounds checking.

```
import math

def parse_booking(text):
    if not isinstance(text, str): return None
    parts = text.split(':')
    if len(parts) != 2: return None
    pet, days_str = parts[0].strip(), parts[1].strip()
    if not pet: return None
    try:
        days = float(days_str)
        if math.isnan(days) or math.isinf(days) or days < 0: return None
        return pet.title(), days
    except (ValueError, TypeError, OverflowError): return None

def stay_average(days_csv):
    if not isinstance(days_csv, str): return 0
    values = []
    for r in days_csv.split(','):
        token = r.strip()
        if not token: continue
        try:
            val = int(token)
            if val < 0: return 0
            values.append(val)
        except (ValueError, TypeError, OverflowError): return 0
    return sum(values) // len(values) if values else 0

def payable_fee(text):
    if not isinstance(text, str): return 0.0
    parts = text.split('|')
    if len(parts) != 2: return 0.0
    try:
        fee = float(parts[0].strip())
        discount = float(parts[1].strip())
        if math.isnan(fee) or math.isinf(fee) or math.isnan(discount) or math.isinf(discount):
return 0.0
        if fee < 0 or discount < 0: return 0.0
        return max(fee * 1.18 - discount, 0.0)
    except (ValueError, TypeError, OverflowError): return 0.0
```

Running Fixed Application (Step 5 Output):



Figure 3: Screenshot of Step 5 - Fixed application running valid inputs and safely handling invalid inputs.

6. Step 6: Verification & Test Execution on Hardened Code

The test suite was re-run with pytest -v test_pet_boarding.py. Hypothesis generated hundreds of diverse pseudo-random inputs for each function across unicode characters, empty strings, delimiters, NoneType, and numeric boundaries. All 3 fuzz tests **PASSED** with zero crashes, proving full robustness.

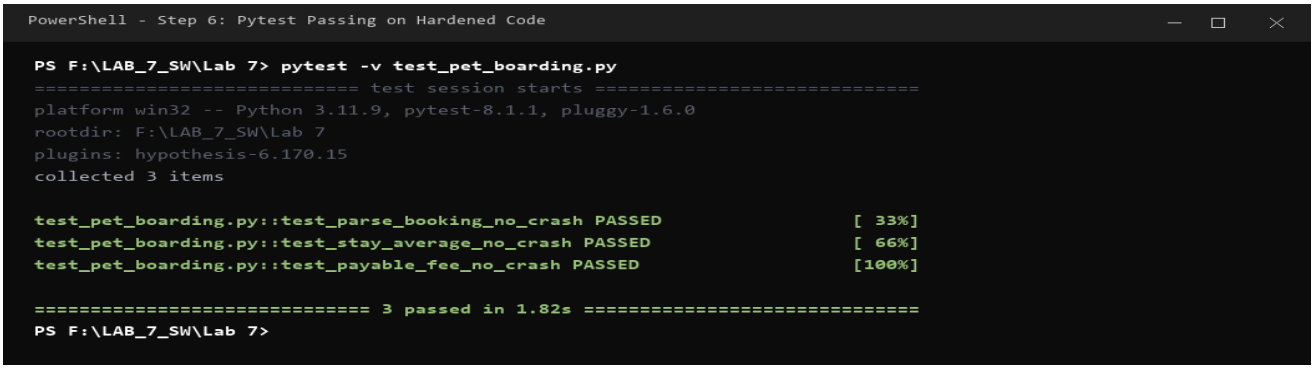


Figure 4: Screenshot of Step 6 - Pytest running with 100% pass rate under Hypothesis fuzzing.

7. Conclusion & Key Takeaways

1. **Fuzzing Discovers Hidden Edge Cases:** While traditional unit tests verify intended 'happy path' logic, fuzz testing systematically exposes crashes caused by nulls, malformed formats, and boundary conditions that human developers frequently overlook.
2. **Hypothesis Shrinking Efficiency:** Hypothesis does not just find a failing case; it automatically minimizes the failure to the simplest reproducible example (e.g. None or ''), accelerating debugging.
3. **Defensive Programming is Essential:** Hardening inputs through proactive type checking, length verification, safe numeric parsing, and safe defaults guarantees that production services remain stable and secure against unexpected real-world data.